TICKLED PINK

Club Entry & Ticketing System

Product Requirements Document — TestSprite QA Submission

Document Version: 1.0 | Platform: Web (Mobile-first) | Market: India

Stack: Node.js + Express + PostgreSQL + Razorpay + Twilio WhatsApp

1. Product Overview

Tickled Pink is a real-time club entry and ticketing system built for Indian nightclubs. It replaces slow, paper-based door management with a digital end-to-end flow: guests register online, pay via Razorpay (UPI / Card / Wallets), receive a QR-coded entry pass on WhatsApp, and are scanned at the entrance by a bouncer.

The system is designed to handle a venue of up to 300 guests per night across multiple entry gates simultaneously. All devices — guest phones, bouncer tablets, admin laptops — connect to a single PostgreSQL backend, ensuring every gate sees live data in real time.

1.1 Target Users

User Role	Description
Guest	Person attending the event. Uses their own phone to register, pay, and receive a WhatsApp entry pass.
Bouncer	Staff at the entrance gate. Uses a tablet or phone to scan guest QR codes and verify entry.
Admin / Manager	Club manager. Views live dashboard, manages the guest list, exports data, and clears event records.

1.2 Core Problem Solved

Traditional club entry involves long queues, cash handling, manual checks, and zero data about who is inside. Tickled Pink eliminates all of these:

- Guests pre-register and pre-pay — no cash at the door
- QR scan at entry takes under 3 seconds per person
- Admin sees live headcount, revenue, and capacity in real time
- Every action is logged — full audit trail for disputes

1.3 Key Constraints

Constraint	Detail
Indian market only	Aadhaar number collected for identity. WhatsApp numbers are Indian (+91). Razorpay handles INR payments.
18+ age enforcement	Every member of a party must be 18 or older. System collects individual ages and blocks the entire group if any person is underage.

Mobile-first	Guests use personal phones. Bouncers use phones/tablets. Admin may use laptop. All views must work on small screens.
Real-time multi-device	Multiple scan gates must share live data. A token scanned at Gate 1 is immediately marked used at Gate 2.
Event-scoped data	Each night is a separate event. Records are scoped by date. Admin can clear one night's data without affecting guest list.

2. User Flows

2.1 Guest Check-In Flow (4 Steps)

The entire guest journey from opening the app to receiving their entry pass happens in 4 sequential steps.

Step 1 — Identity Collection

Guest fills in their details on Step 1 of the check-in form:

- Full name
- WhatsApp number (10 digits, Indian mobile)
- Aadhaar number (12 digits, auto-formatted as XXXX XXXX XXXX)
- Party size (1 to 5 people)
- Age of every person in the party (individual boxes, all must be 18-99)

Validation rules enforced at this step:

- Name: required, max 100 characters
- WhatsApp: exactly 10 digits, Indian mobile format
- Aadhaar: 12 digits, first digit must be 2-9 (official Aadhaar format rule)
- Ages: whole numbers only (no decimals), each 18-99, any person under 18 blocks the entire group
- Duplicate WhatsApp: blocked with error — "already checked in tonight"
- Duplicate Aadhaar: blocked with error — "already checked in tonight"
- Venue at capacity: form blocked with capacity banner

Step 2 — Ticket Selection

Guest selects their ticket type. Prices are shown per person and as total for the party size entered in Step 1.

Ticket Type	Price per Person	Who Can Select
General	₹500	Anyone
VIP	₹1,500	Anyone — priority lane + drinks included
Guest List	₹200	Pre-approved guests only — requires name + WhatsApp verification against admin-loaded list

Guest List verification: when selected, guest must enter their registered WhatsApp number AND exact name. Both must match the admin's pre-loaded list. Party size cannot exceed the allowed party size set by the admin for that guest.

Step 3 — Payment

Guest pays via Razorpay Checkout. Supported payment methods:

- UPI (PhonePe, Google Pay, Paytm, any UPI app)
- Credit / Debit Card
- Net Banking
- Wallets

Total amount is always computed server-side (ticket price × party size). The client-side display is for reference only — the server generates the Razorpay order and the amount is locked there, not in the browser.

Payment flow: Frontend requests order from server → Server creates Razorpay order → Frontend opens Razorpay Checkout popup → Guest pays → Razorpay returns payment ID + signature → Server verifies HMAC signature → Check-in proceeds.

Step 4 — Entry Pass

After successful payment verification, the system:

- Generates a unique 6-character entry token (e.g. A3K9X2) — collision-checked against database
- Saves full guest record to PostgreSQL database
- Renders QR code on screen immediately (guest can screenshot)
- Sends WhatsApp message to guest's number via Twilio with token details
- Updates admin dashboard stats in real time

WhatsApp message content:

- Club name + "Entry Confirmed"
- Guest name, token, ticket type, party size, amount paid
- Entry window (doors open time to close time)
- Reminder: show QR at entrance, photo ID required

2.2 Bouncer Scan Flow

Bouncer opens the Scan tab using their Bouncer PIN (4-digit, separate from Admin PIN). The scan tab has two entry methods:

Camera QR Scan

- Bouncer taps "Start Camera" — phone/tablet camera opens
- Camera waits for full initialization before beginning to scan (loadedmetadata event — prevents false "not working" perception)

- jsQR library scans frames at 300ms intervals — works on all browsers including iOS Safari and Firefox
- BarcodeDetector API used as enhancement on Chrome/Android only (requires HTTPS)
- Both normal and inverted QR codes detected (attemptBoth mode — handles dark-background QR codes)
- On successful decode: token extracted → API call made → result shown

Manual Token Entry

- Bouncer types 6-character token manually (fallback for damaged QR)
- Input field cleared automatically after each verify to prevent accidental double-verify

Scan Results

Six possible outcomes displayed with distinct colors and clear labels:

Result	Color	Meaning
Valid Entry	Green	Token found, not yet scanned, within entry window. Shows name, party count, ticket type. Bouncer lets in the stated number of people.
Already Scanned	Yellow	Token was used earlier. Shows original scan time. Bouncer should deny entry unless re-entry mode is ON.
Re-Entry	Yellow (RE-ENTRY label)	Re-entry mode is active, token already scanned. Shows original scan time. Bouncer allows re-entry.
Invalid Token	Red	Token not found in database. Could be forged or wrong event.
Doors Closed	Red	Current time is outside the configured entry window. Token may be valid but cannot be used.
Expired Pass	Red	Token was issued on a different date (previous event). Cannot be used tonight.

Re-Entry Mode

Toggle switch on the scan tab. When ON: already-scanned tokens show a yellow "Re-Entry" result instead of blocking. Used for guests who step outside to smoke and re-enter. When OFF: already-scanned tokens are blocked.

2.3 Admin Dashboard Flow

Admin opens dashboard using the Admin PIN (full access — Admin PIN also opens Bouncer scan tab). Dashboard shows:

Live Statistics

Stat	How Computed	Updates When
People Inside	Count of scanned guests (sum of party sizes)	Every token scan
VIP / General / Guest List	Breakdown by ticket type (sum of party sizes per type)	Every check-in
Total Bookings	Count of guest records in DB for tonight	Every check-in
Revenue	Sum of price_total for tonight	Every check-in
Flags	Count of duplicate_attempt entries in audit_log	Every blocked duplicate
Yet to Enter	Guests checked in but not yet scanned at door	Every check-in and scan

Capacity Bar

Visual progress bar showing current headcount vs maximum capacity (configurable, default 300). Bar colour changes: green below 80%, yellow 80-94%, red at 95%+.

Guest Table

Searchable table of all check-ins for tonight. Search works on name, token, and masked WhatsApp number. Columns: Token, Name, WhatsApp (masked to last 4 digits), Aadhaar (masked — XXXX XXXX + last 4), Type, Party size, Amount paid, Check-in time, Status (Waiting / Entered).

CSV Export

Downloads a CSV file of all tonight's check-ins. Contains the same columns as the table. WhatsApp and Aadhaar are masked in the export — full numbers are never exported.

Clear Event Data

Requires a confirmation dialog. Deletes all guest check-in records for tonight. Guest list is preserved — it carries over across events. Action is logged to audit_log before deletion.

2.4 Guest List Management Flow

Admin adds pre-approved guests before the event. Each entry has:

- Full name (must match exactly at check-in)
- WhatsApp number (must match exactly at check-in)
- Allowed party size (admin sets how many people this guest can bring)

Status lifecycle: Waiting → Checked In. Status updates automatically when the guest completes check-in. Admin can also manually mark a guest as "Arrived" (for cases where the number was entered slightly wrong but identity is confirmed in person).

Remove: admin can remove waiting guests. Cannot remove already checked-in guests. Confirmation dialog required before removal.

3. Feature Specifications

3.1 Authentication & Access Control

Feature	Specification
Admin PIN	4-digit PIN. Stored as bcrypt hash (cost 12) in server environment. Never in frontend code. Grants access to: Admin dashboard, Guest List, Scan tab.
Bouncer PIN	Separate 4-digit PIN. Also bcrypt hashed. Grants access to Scan tab only. Cannot open Admin or Guest List.
JWT Tokens	Issued on successful PIN verify. Expiry: 8 hours (one event night). Contains role field (admin or bouncer). Stored in JavaScript memory only — never localStorage or sessionStorage.
Auth Failure Handling	On 401 or 403 from any API: clear JWT from memory, show PIN overlay automatically. Bouncer never sees a confusing blank screen.
Rate Limiting	Maximum 5 PIN attempts per IP address per minute. Returns HTTP 429 with "wait 1 minute" message after limit hit.
Session Expiry	After 8 hours, next API call returns 401, PIN overlay re-appears automatically.

3.2 Identity Collection & Validation

Field	Validation Rules
Full Name	Required. Trimmed. Maximum 100 characters. Stored as entered.
WhatsApp Number	Required. Exactly 10 digits. Indian mobile number. Stored as raw digits (no country code). Masked in all API responses and exports.
Aadhaar Number	Required. 12 digits. First digit must be 2-9 (official Aadhaar specification). Stored as raw 12 digits (spaces stripped). Never returned in any API response. Masked in admin table (XXXX XXXX + last 4).
Ages	One input per party member. Whole numbers only (no decimals — step=1 attribute + Number.isInteger() check). Range 18-99. Any member under 18 blocks the entire group with a specific error naming which person failed.
Party Size	Dropdown: 1 to 5 people. Age inputs update dynamically. Labels: "You", "Guest 1", "Guest 2" etc.
Duplicate Check	WhatsApp number checked against all tonight's records. Aadhaar checked independently. Both checks happen server-side using parameterized SQL queries. Client-side check is a UX optimization only.

3.3 Payment Integration (Razorpay)

Two modes controlled by environment variable:

Mode	Behaviour
Dev Mode (DEV_MODE=true)	Order created as mock. Payment auto-completes instantly. No real Razorpay popup. Used for testing and development.
Production Mode (DEV_MODE=false)	Real Razorpay Checkout popup opens. Guest pays via UPI/Card/etc. Server verifies HMAC signature using Razorpay secret key before proceeding to check-in.

Security: Total amount is computed server-side from ticket type × party size. The Razorpay order is created with the server-computed amount. Client cannot manipulate the price by editing the DOM.

3.4 WhatsApp Notifications (Twilio)

Aspect	Detail
Provider	Twilio WhatsApp API. Sandbox mode for testing (guests must opt in with join keyword). Production mode uses approved WhatsApp Business number.
Trigger	Automatically called after successful check-in. Never blocks check-in if WA delivery fails — failure is logged and waStatus:"failed" returned in response.
Message Content	Club name, confirmation header, guest name, 6-char token, ticket type, party count, amount paid (INR formatted), entry window times, reminder to show QR and bring photo ID.
Resend	Resend endpoint available. Guest or staff can trigger re-delivery. Looks up guest by token from database (not from browser memory).
Template Approval	For first-contact messages (guests who have never WhatsApped the club), a pre-approved Twilio message template is required by Meta. Template must be submitted and approved before go-live.
Mock Mode	If Twilio credentials not configured, message is logged to console only. System continues to work — WhatsApp is a notification layer, not a gating requirement.

3.5 QR Code Generation & Scanning

Aspect	Detail
QR Content	JSON string containing: token, name, ticket type, party size, timestamp, event date. Error correction level H (30% damage tolerance).
Display	Rendered client-side using QRCode.js library. Shown on Step 4 after payment. Spinner shown while QR renders — pass revealed only after canvas element is confirmed in DOM (prevents blank screenshot issue).
WhatsApp Delivery	Text-only message in current version. QR shown on screen for guests to screenshot. Production upgrade path: generate QR server-side with qrcode npm package, upload to Supabase Storage, send image URL via Twilio mediaUrl.
Scan Library	jsQR (primary — works on all browsers, HTTP and HTTPS, including iOS Safari). BarcodeDetector API (enhancement — Chrome/Android only, requires HTTPS).
Inversion Handling	attemptBoth mode — detects both normal (dark code on light background) and inverted (light code on dark background) QR codes.
Camera Init	Scan loop starts only after loadedmetadata event fires — ensures camera stream is fully ready before scanning begins. Eliminates "scanner not working" perception on slow mobile cameras.

3.6 Capacity & Entry Window Controls

Control	Behaviour
Max Capacity	Configurable (default: 300 people). Enforced server-side at check-in — returns 400 error if adding the party would exceed capacity. Enforced client-side as UX — shows "AT CAPACITY" banner and hides the form. Capacity bar in admin turns red at 95%+.
Doors Open / Close	Configurable time range (e.g. 21:00 to 03:00). Crosses midnight correctly. Check-in form blocked outside window. Scanner shows "DOORS CLOSED" result outside window. Dev mode bypasses time check for testing.
Dev Mode	DEV_MODE=true: doors always open, Razorpay mock, time check bypassed. Set false for production. Status badge in scan tab shows whether doors are currently open or closed.

4. API Reference

Base URL: http://localhost:3000 (development) / https://[app].up.railway.app (production). All request and response bodies are JSON. All protected endpoints require Authorization: Bearer {jwt} header.

4.1 Public Endpoints (No Authentication)

Method	Endpoint	Purpose	Key Response Fields
GET	/api/health	Server health check	status, timestamp, dbConnected
GET	/api/auth/config	Club config for frontend	clubName, eventName, maxCapacity, tickets[]
POST	/api/auth/verify	PIN → JWT token	token, role, expiresIn
POST	/api/payments/create-order	Create Razorpay order	orderId, amount, keyId, mock
POST	/api/payments/verify	Verify payment signature	ok, paymentId
POST	/api/guests/checkin	Complete check-in after payment	token, qrData, waStatus
POST	/api/guestlist/verify	Verify guest list membership	verified, reason, partyAllowed
POST	/api/guests/resend/:token	Resend WhatsApp pass	sent, wa (masked), error

4.2 Bouncer Endpoints (Bouncer or Admin JWT)

Method	Endpoint	Purpose	Key Response Fields
GET	/api/guests/verify/:token	Look up token info	found, guest{name, type, party, scanned}
POST	/api/guests/scan/:token	Mark token scanned	result: ok|already_scanned|reentry|not_found|expired|closed

4.3 Admin Endpoints (Admin JWT Only)

Method	Endpoint	Purpose	Key Response Fields

GET	/api/guests	All guests (searchable)	guests[], total
GET	/api/guests/stats	Live dashboard stats	peopleInside, vip, revenue, capacityPct
GET	/api/guests/export	CSV file download	File attachment
DELETE	/api/guests/clear	Clear tonight's records	cleared (count), date
GET	/api/guestlist	Full guest list	guestList[]{id, name, wa_masked, partyAllowed, status}
POST	/api/guestlist	Add guest to list	id, name, wa_masked, partyAllowed, status
DELETE	/api/guestlist/:id	Remove guest	deleted: true or error: already_checked_in
POST	/api/guestlist/:id/arrive	Mark guest arrived manually	id, status: checked-in
GET	/api/guestlist/export	Guest list CSV download	File attachment

5. Database Schema

PostgreSQL hosted on Supabase. Three tables. All queries use parameterized placeholders ($1, $2…) — no string interpolation in SQL.

5.1 guests

One record per check-in transaction. Covers one booking (may include multiple people via party size).

Column	Type / Constraints
id	SERIAL PRIMARY KEY
token	VARCHAR(6) UNIQUE NOT NULL — 6-char alphanumeric, collision-safe
name	TEXT NOT NULL
wa	VARCHAR(10) NOT NULL — raw 10 digits, no country code
aadhar	VARCHAR(12) NOT NULL — raw 12 digits, no spaces
type	VARCHAR(20) NOT NULL — general | vip | guestlist
type_name	VARCHAR(20) NOT NULL — display label
party	INTEGER NOT NULL DEFAULT 1
ages	JSONB NOT NULL DEFAULT [] — array of integers
price_per_person	INTEGER NOT NULL — in rupees (not paise)
price_total	INTEGER NOT NULL — price_per_person * party
payment_id	TEXT — Razorpay payment ID, null in dev mode
order_id	TEXT — Razorpay order ID, null in dev mode
scanned	BOOLEAN NOT NULL DEFAULT false
scan_time	TIMESTAMPTZ — null until scanned
event_date	DATE NOT NULL DEFAULT CURRENT_DATE
created_at	TIMESTAMPTZ NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ NOT NULL DEFAULT NOW()

Indexes: token (unique), wa, aadhar, event_date.

5.2 guest_list

Persistent across events. Admin manages this before each event night.

Column	Type / Constraints
id	SERIAL PRIMARY KEY
name	TEXT NOT NULL
wa	VARCHAR(10) UNIQUE NOT NULL
party_allowed	INTEGER NOT NULL DEFAULT 1 — max party size allowed for this guest
status	VARCHAR(20) NOT NULL DEFAULT waiting — waiting | checked-in
created_at	TIMESTAMPTZ NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ NOT NULL DEFAULT NOW()

5.3 audit_log

Immutable append-only log. Every significant action recorded with role and IP.

Column	Type / Constraints
id	SERIAL PRIMARY KEY
action	VARCHAR(50) NOT NULL
details	JSONB — action-specific data
role	VARCHAR(20) — admin | bouncer | system
ip	TEXT — request IP address
created_at	TIMESTAMPTZ NOT NULL DEFAULT NOW()

Logged actions: checkin, scan, reentry, duplicate_attempt, payment_verified, payment_mock, clear_event, auth_fail, guestlist_add, guestlist_remove, guestlist_arrive.

6. Test Scenarios

The following scenarios must all pass before the system is considered production-ready. TestSprite should cover all happy paths, all error paths, and all edge cases listed here.

6.1 Check-In Happy Path

1. Open app → Step 1 form is visible, no banners
2. Fill name, valid 10-digit WA, valid Aadhaar starting with 2-9, party size 2, ages 21 and 24
3. Click Continue → advances to Step 2 ticket selection
4. Select General ticket → price shows ₹1,000 (2 × ₹500)
5. Click Proceed → Step 3 shows ₹1,000 amount
6. Click Pay → Razorpay opens (dev mode: auto-completes)
7. Step 4: QR code renders, token shown, pass details correct
8. WhatsApp message delivered to entered number (or console.log in mock mode)

6.2 Validation Error Cases

Test Input	Expected Error Message	Expected Behaviour
Empty name field	Enter your full name.	Stay on Step 1, error banner shown
WhatsApp: 9 digits	Enter a valid 10-digit WhatsApp number.	Stay on Step 1
Aadhaar starting with 1	Invalid Aadhaar — 12 digits, first digit 2-9.	Stay on Step 1
Age field: 17	You must be 18+. Entire group denied.	Stay on Step 1
Age field: 17.5 (decimal)	Enter a whole number age for: yourself.	Step attribute prevents input; validation catches it
Age field: 100	Age cannot exceed 99 for: yourself.	Stay on Step 1
Guest 2 age: 16 (party of 2)	Guest 1 must be 18+. Entire group denied.	Names specific guest
Duplicate WA (second attempt)	This WhatsApp is already checked in.	Duplicate count incremented in audit_log
Duplicate Aadhaar (second attempt)	This Aadhaar is already checked in.	Duplicate count incremented
At capacity (300 people already in)	Venue at capacity — cannot add N more.	Shows capacity banner on check-in page

6.3 Guest List Verification Cases

Test Case	Expected Result	Notes
Correct WA + correct name (case-insensitive)	Verified — proceed to payment	Both must match
Correct WA + wrong name	No match — check exact name and number.	Both required
Wrong WA + correct name	No match — check exact name and number.	Both required
Correct WA + name, but already checked in	This guest already checked in.	Status check
Party size 3, guest allowed 2	Party of 3 exceeds allowed 2.	Party size enforced
Guest list ticket, no verification attempt	Alert: Verify guest list first.	Cannot skip

6.4 Scanner Test Cases

Test Case	Expected Result	Display
Valid token, first scan, within door hours	ok	Green — "Let in N people · Type"
Same valid token scanned again (re-entry OFF)	already_scanned	Yellow — shows first scan time
Same valid token (re-entry mode ON)	reentry	Yellow — RE-ENTRY MODE ACTIVE label
Token not in database	not_found	Red — INVALID TOKEN
Token from yesterday	expired	Red — EXPIRED PASS with date
Any token, outside door hours (devMode=false)	closed	Red — DOORS CLOSED with window
Expired JWT during scan	401 from API	PIN overlay opens automatically
No JWT (not authenticated)	Immediate redirect	PIN overlay opens before API call
Manual token entry: correct token	ok	Green result, input clears after
Manual token entry: wrong token	not_found	Red result, input clears after

Camera scan: inverted QR (dark background)	Detected and verified	jsQR attemptBoth mode handles this
Camera scan: iPhone Safari	Camera opens, jsQR engine used	Engine label shows "jsQR"
Camera scan: Android Chrome	Camera opens, native or jsQR used	Engine label shows engine name

6.5 Admin Dashboard Cases

9. Open Admin tab → PIN overlay shown
10. Enter Admin PIN → dashboard loads with correct stats
11. Stats update after each check-in without page refresh
12. Search by partial name → table filters correctly
13. Search by token → single record shown
14. Capacity bar at 0% → green
15. Simulate 240 check-ins (80%) → bar turns yellow
16. Simulate 285 check-ins (95%) → bar turns red
17. Export CSV → file downloads with masked WA and Aadhaar, all records present
18. Clear Event Data → confirmation dialog shown → on confirm: records gone, guest list preserved
19. Clear Event Data → cancel → records unchanged

6.6 Multi-Device Sync Cases

20. Check in guest on Device A → open Admin on Device B → guest appears in table
21. Scan token on Device A (Gate 1) → scan same token on Device B (Gate 2) → already_scanned result on Device B
22. Admin clears event on Device A → bouncer on Device B scans any token → not_found (records cleared)

6.7 Security Cases

23. Enter Admin PIN in Bouncer PIN field → 401, PIN overlay stays open
24. Enter wrong PIN 5 times in 60 seconds → HTTP 429, rate limit message shown
25. Inspect page source → no PINs visible, no JWT secret visible
26. Modify DOM price to ₹1 before payment → Razorpay order uses server-computed amount, not DOM value
27. Make API call to /api/guests with no Authorization header → 401 returned
28. Make API call with Bouncer JWT to /api/guests/stats (admin-only) → 403 returned
29. SQL injection attempt in name field: '; DROP TABLE guests; -- → rejected by parameterized query, stored as literal string

7. Technology Stack

Layer	Technology & Version
Runtime	Node.js 18+
Framework	Express.js 4.x
Database	PostgreSQL (hosted on Supabase free tier)
DB Driver	pg (node-postgres) 8.x — connection pool, parameterized queries
Authentication	jsonwebtoken 9.x — JWT, 8-hour expiry, role-based
PIN Hashing	bcrypt 5.x — cost factor 12, salted
Payments	Razorpay Node SDK 2.x — HMAC signature verification
WhatsApp	Twilio 5.x — WhatsApp Business API, sandbox + production modes
QR Generation	qrcode 1.x — server-side generation, error correction level H
QR Scanning	jsQR 1.4.x (primary, all browsers) + BarcodeDetector API (Chrome enhancement)
Security Headers	helmet 7.x
Rate Limiting	express-rate-limit 7.x
CORS	cors 2.x — restricted to known origin in production
Frontend	Single HTML file — Vanilla JS, no framework dependency
Hosting	Railway.app — persistent filesystem, managed PostgreSQL, auto-deploy from GitHub

8. Environment Configuration

All configuration lives in a .env file on the server. Nothing sensitive is shipped to the browser. The frontend loads non-sensitive config from GET /api/auth/config at startup.

Variable	Purpose
PORT	Server port (default 3000)
NODE_ENV	development or production — affects SSL mode, CORS behaviour
DATABASE_URL	Full PostgreSQL connection string from Supabase
ADMIN_PIN_HASH	bcrypt hash of Admin PIN — generated by scripts/hash-pins.js

BOUNCER_PIN_HASH	bcrypt hash of Bouncer PIN
JWT_SECRET	Minimum 32 random characters — signs and verifies all JWT tokens
RAZORPAY_KEY_ID	Razorpay API key ID (rzp_test_... or rzp_live_...)
RAZORPAY_KEY_SECRET	Razorpay API secret — used for HMAC signature verification
TWILIO_ACCOUNT_SID	Twilio account identifier (starts with AC)
TWILIO_AUTH_TOKEN	Twilio auth token
TWILIO_WA_FROM	WhatsApp sender number e.g. whatsapp:+14155238886
CLUB_NAME	Club display name e.g. TICKLED PINK
EVENT_NAME	Event display name e.g. NEON NIGHT — SATURDAY
MAX_CAPACITY	Integer — maximum total people allowed in venue
MAX_PARTY_SIZE	Integer — maximum people per booking (default 5)
DOORS_OPEN	24h time e.g. 21:00
DOORS_CLOSE	24h time e.g. 03:00 (midnight crossing handled automatically)
DEV_MODE	true = mock payments, doors always open. false = production
ALLOWED_ORIGIN	CORS allowed origin e.g. https://app.up.railway.app

9. Known Limitations & Future Work

Limitation	Planned Resolution
WhatsApp sends text only — no QR image	Upgrade: generate QR server-side → upload to Supabase Storage → send image URL via Twilio mediaUrl
Twilio sandbox requires opt-in from guests	Production: upgrade to real WhatsApp Business number with approved message template
No real-time push updates on admin dashboard	Upgrade: add Server-Sent Events or WebSocket so stats refresh without page reload
No per-event reporting across multiple nights	Upgrade: add date filter to admin dashboard, historical reports
No refund flow	Out of scope for MVP — refunds handled manually via Razorpay dashboard
Single-language (English)	Future: Hindi UI option for staff-facing screens
No offline mode for bouncer	Future: service worker cache of token list for local verification during internet outages

10. Glossary

Term	Definition
Token	6-character alphanumeric code (e.g. A3K9X2) generated per booking. Used by bouncer to verify entry. Unique per event night.
QR Code	Machine-readable code encoding the token as a JSON string. Generated after payment. Scanned at the entrance.
Aadhaar	Indian government-issued 12-digit identity number. Collected to prevent identity fraud. Never returned by any API.
Party	Group of people on a single booking. One QR code / token covers the whole party. Bouncer admits the stated party count on a single scan.
Guest List	Pre-approved list of VIP guests uploaded by admin. These guests pay a discounted rate (₹200) but must verify their identity against the list at check-in.
JWT	JSON Web Token. Short-lived cryptographic token (8 hours) that proves the bouncer or admin has authenticated with the correct PIN. Stored in JavaScript memory only.
bcrypt	Password hashing algorithm designed to be slow and salted. Used to store Admin and Bouncer PINs securely on the server.

HMAC	Hash-based Message Authentication Code. Used by Razorpay to sign payment responses. Server verifies this signature to confirm payment is genuine.
Re-entry	When a guest exits the venue temporarily and returns. Re-entry mode on the scan tab allows already-scanned tokens to pass again.
Dev Mode	Configuration flag (DEV_MODE=true) that bypasses door hours, uses mock payments, and logs WhatsApp to console instead of sending.
Audit Log	Immutable database table recording every significant system action with timestamp, role, and IP address.
WATI	WhatsApp API provider (alternate to Twilio). Requires Meta-approved WhatsApp Business Account. Current implementation uses Twilio.